that

$$\mathbf{y} = \mathbf{L}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \tag{7.4.7}$$

has uncorrelated components, each of unit variance. Proof:

$$\begin{aligned} \langle \mathbf{y} \otimes \mathbf{y} \rangle &= \left\langle (\mathbf{L}^{-1}[\mathbf{x} - \boldsymbol{\mu}]) \otimes (\mathbf{L}^{-1}[\mathbf{x} - \boldsymbol{\mu}]) \right\rangle \\ &= \mathbf{L}^{-1} \left\langle (\mathbf{x} - \boldsymbol{\mu}) \otimes (\mathbf{x} - \boldsymbol{\mu}) \right\rangle \mathbf{L}^{-1T} \\ &= \mathbf{L}^{-1} \boldsymbol{\Sigma} \mathbf{L}^{-1T} = \mathbf{L}^{-1} \mathbf{L} \mathbf{L}^T \mathbf{L}^{-1T} = \mathbf{1} \end{aligned} \tag{7.4.8}$$

Be aware that this linear combination is not unique. In fact, once you have obtained a vector $\mathbf{y}$ of uncorrelated components, you can perform any rotation on it and still have uncorrelated components. In particular, if $\mathbf{K}$ is an orthogonal matrix, so that

$$\mathbf{K}^T \mathbf{K} = \mathbf{K} \mathbf{K}^T = \mathbf{1} \tag{7.4.9}$$

then

$$\langle (\mathbf{K}\mathbf{y}) \otimes (\mathbf{K}\mathbf{y}) \rangle = \mathbf{K} \langle \mathbf{y} \otimes \mathbf{y} \rangle \mathbf{K}^T = \mathbf{K} \mathbf{K}^T = \mathbf{1} \tag{7.4.10}$$

A common (though slower) alternative to Cholesky decomposition is to use the Jacobi transformation (§11.1) to decompose $\boldsymbol{\Sigma}$ as

$$\boldsymbol{\Sigma} = \mathbf{V} \mathrm{diag}(\sigma_i^2) \mathbf{V}^T \tag{7.4.11}$$

where $\mathbf{V}$ is the orthogonal matrix of eigenvectors, and the σ_i's are the standard deviations of the (new) uncorrelated variables. Then $\mathbf{V}\mathrm{diag}(\sigma_i)$ plays the role of $\mathbf{L}$ in the proofs above.

Section §16.1.1 discusses some further applications of Cholesky decomposition relating to multivariate random variables.

7.5 Linear Feedback Shift Registers

A *linear feedback shift register* (LFSR) consists of a *state vector* and a certain kind of *update rule*. The state vector is often the set of bits in a 32- or 64-bit word, but it can sometimes be a set of words in an array. To qualify as an LFSR, the update rule must generate a *linear* combination of the bits (or words) in the current state, and then shift that result onto one end of the state vector. The oldest value, at the other end of the state vector, falls off and is gone. The output of an LFSR consists of the sequence of new bits (or words) as they are shifted in.

For single bits, "linear" means arithmetic modulo 2, which is the same as using the logical XOR operation for $+$ and the logical AND operation for $\times$. It is convenient, however, to write equations using the arithmetic notation. So, for an LFSR of length n, the words in the paragraph above translate to

$$\begin{aligned} a'_1 &= \left(\sum_{j=1}^{n-1} c_j a_j \right) + a_n \\ a'_i &= a_{i-1}, \qquad i = 2, \ldots, n \end{aligned} \tag{7.5.1}$$

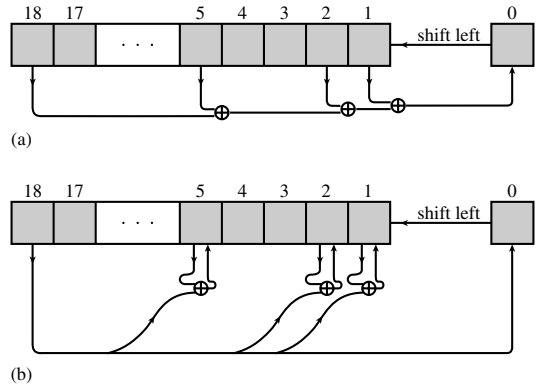

Figure 7.5.1. Two related methods for obtaining random bits from a shift register and a primitive polynomial modulo 2. (a) The contents of selected taps are combined by XOR (addition modulo 2), and the result is shifted in from the right. This method is easiest to implement in hardware. (b) Selected bits are modified by XOR with the leftmost bit, which is then shifted in from the right. This method is easiest to implement in software.

Here $\mathbf{a}'$ is the new state vector, derived from $\mathbf{a}$ by the update rule as shown. The reason for singling out a_n in the first line above is that its coefficient c_n must be $\equiv 1$. Otherwise, the LFSR wouldn't be of length n, but only of length up to the last nonzero coefficient in the c_j's.

There is also a reason for numbering the bits (henceforth we consider only the case of a vector of bits, not of words) starting with 1 rather than the more comfortable 0. The mathematical properties of equation (7.5.1) derive from the properties of the polynomials over the integers modulo 2. The polynomial associated with (7.5.1) is

$$P(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_2x^2 + c_1x + 1 \tag{7.5.2}$$

where each of the c_i's has the value 0 or 1. So, c_0, like c_n, exists but is implicitly $\equiv 1$. There are several notations for describing specific polynomials like (7.5.2). One is to simply list the values i for which c_i is nonzero (by convention including c_n and c_0). So the polynomial

$$x^{18} + x^5 + x^2 + x + 1 \tag{7.5.3}$$

is abbreviated as

$$(18, 5, 2, 1, 0) \tag{7.5.4}$$

Another, when a value of n (here 18), and $c_n = c_0 = 1$, is assumed, is to construct a "serial number" from the binary word $c_{n-1}c_{n-1}\cdots c_2c_1$ (by convention now excluding c_n and c_0). For (7.5.3) this would be 19, that is, $2^4 + 2^1 + 2^0$. The nonzero c_i's are often referred to as an LFSR's *taps*.

Figure 7.5.1(a) illustrates how the polynomial (7.5.3) and (7.5.4) looks as an update process on a register of 18 bits. Bit 0 is the temporary where a bit that is to become the new bit 1 is computed.

The maximum period of an LFSR of n bits, before its output starts repeating, is $2^n - 1$. This is because the maximum number of distinct states is 2^n, but the special vector with all bits zero simply repeats itself with period 1. If you pick a random polynomial $P(x)$, then the generator you construct will usually not be full-period. A

fraction of polynomials over the integers modulo 2 are *irreducible*, meaning that they can't be factored. A fraction of the irreducible polynomials are *primitive*, meaning that they generate maximum period LFSRs. For example, the polynomial $x^2 + 1 = (x + 1)(x + 1)$ is not irreducible, so it is not primitive. (Remember to do arithmetic on the coefficients mod 2.) The polynomial $x^4 + x^3 + x^2 + x + 1$ is irreducible, but it turns out not to be primitive. The polynomial $x^4 + x + 1$ is both irreducible and primitive.

Maximum period LFSRs are often used as sources of random bits in hardware devices, because logic like that shown in Figure 7.5.1(a) requires only a few gates and can be made to run extremely fast. There is not much of a niche for LFSRs in software applications, because implementing equation (7.5.1) in code requires at least two full-word logical operations for each nonzero c_i, and all this work produces a meager one bit of output. We call this "Method I." A better software approach, "Method II," is not obviously an LFSR at all, but it turns out to be mathematically equivalent to one. It is shown in Figure 7.5.1(b). In code, this is implemented from a primitive polynomial as follows:

Let `maskp` and `maskn` be two bit masks,

$$\begin{array}{rl} \texttt{maskp} \equiv & (0 \quad \cdots \quad 0 \quad c_{n-1} \quad c_{n-2} \quad \cdots \quad c_2 \quad c_1) \\ \texttt{maskn} \equiv & (0 \quad \cdots \quad 1 \quad 0 \quad 0 \quad \cdots \quad 0 \quad 0) \end{array} \tag{7.5.5}$$

Then, a word **a** is updated by

```
if (a & maskn) a = ((a ^ maskp) << 1) | 1;
else a <<= 1;
```
(7.5.6)

You should work through the above prescription to see that it is identical to what is shown in the figure. The output of this update (still only one bit) can be taken as `(a & maskn)`, or for that matter any fixed bit in `a`.

LFSRs (either Method I or Method II) are sometimes used to get random m-bit words by concatenating the output bits from m consecutive updates (or, equivalently for Method I, grabbing the low-order m bits of state after every m updates). This is generally a bad idea, because the resulting words usually fail some standard statistical tests for randomness. It is especially a bad idea if m and $2^n - 1$ are not relatively prime, in which case the method does not even give all m-bit words uniformly.

Next, we'll develop a bit of theory to see the relation between Method I and Method II, and this will lead us to a routine for testing whether any given polynomial (expressed as a bit string of c_i's) is primitive. But, for now, if you only need a table of some primitive polynomials go get going, one is provided on the next page.

Since the update rule (7.5.1) is linear, it can be written as a matrix $\mathbf{M}$ that multiplies from the left a column vector of bits **a** to produce an updated state $\mathbf{a}'$. (Note that the low-order bits of **a** start at the top of the column vector.) One can readily read off

$$\mathbf{M} = \begin{bmatrix} c_1 & c_2 & \dots & c_{n-2} & c_{n-1} & 1 \\ 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 & 0 \\ 0 & 0 & \dots & 0 & 1 & 0 \end{bmatrix} \tag{7.5.7}$$

Some Primitive Polynomials Modulo 2 (after Watson [1])

(1, 0)	(51, 6, 3, 1, 0)
(2, 1, 0)	(52, 3, 0)
(3, 1, 0)	(53, 6, 2, 1, 0)
(4, 1, 0)	(54, 6, 5, 4, 3, 2, 0)
(5, 2, 0)	(55, 6, 2, 1, 0)
(6, 1, 0)	(56, 7, 4, 2, 0)
(7, 1, 0)	(57, 5, 3, 2, 0)
(8, 4, 3, 2, 0)	(58, 6, 5, 1, 0)
(9, 4, 0)	(59, 6, 5, 4, 3, 1, 0)
(10, 3, 0)	(60, 1, 0)
(11, 2, 0)	(61, 5, 2, 1, 0)
(12, 6, 4, 1, 0)	(62, 6, 5, 3, 0)
(13, 4, 3, 1, 0)	(63, 1, 0)
(14, 5, 3, 1, 0)	(64, 4, 3, 1, 0)
(15, 1, 0)	(65, 4, 3, 1, 0)
(16, 5, 3, 2, 0)	(66, 8, 6, 5, 3, 2, 0)
(17, 3, 0)	(67, 5, 2, 1, 0)
(18, 5, 2, 1, 0)	(68, 7, 5, 1, 0)
(19, 5, 2, 1, 0)	(69, 6, 5, 2, 0)
(20, 3, 0)	(70, 5, 3, 1, 0)
(21, 2, 0)	(71, 5, 3, 1, 0)
(22, 1, 0)	(72, 6, 4, 3, 2, 1, 0)
(23, 5, 0)	(73, 4, 3, 2, 0)
(24, 4, 3, 1, 0)	(74, 7, 4, 3, 0)
(25, 3, 0)	(75, 6, 3, 1, 0)
(26, 6, 2, 1, 0)	(76, 5, 4, 2, 0)
(27, 5, 2, 1, 0)	(77, 6, 5, 2, 0)
(28, 3, 0)	(78, 7, 2, 1, 0)
(29, 2, 0)	(79, 4, 3, 2, 0)
(30, 6, 4, 1, 0)	(80, 7, 5, 3, 2, 1, 0)
(31, 3, 0)	(81, 4 0)
(32, 7, 5, 3, 2, 1, 0)	(82, 8, 7, 6, 4, 1, 0)
(33, 6, 4, 1, 0)	(83, 7, 4, 2, 0)
(34, 7, 6, 5, 2, 1, 0)	(84, 8, 7, 5, 3, 1, 0)
(35, 2, 0)	(85, 8, 2, 1, 0)
(36, 6, 5, 4, 2, 1, 0)	(86, 6, 5, 2, 0)
(37, 5, 4, 3, 2, 1, 0)	(87, 7, 5, 1, 0)
(38, 6, 5, 1, 0)	(88, 8, 5, 4, 3, 1, 0)
(39, 4, 0)	(89, 6, 5, 3, 0)
(40, 5, 4 3, 0)	(90, 5, 3, 2, 0)
(41, 3, 0)	(91, 7, 6, 5, 3, 2, 0)
(42, 5, 4, 3, 2, 1, 0)	(92, 6, 5, 2, 0)
(43, 6, 4, 3, 0)	(93, 2, 0)
(44, 6, 5, 2, 0)	(94, 6, 5, 1, 0)
(45, 4, 3, 1, 0)	(95, 6, 5, 4, 2, 1, 0)
(46, 8, 5, 3, 2, 1, 0)	(96, 7, 6, 4, 3, 2, 0)
(47, 5, 0)	(97, 6, 0)
(48, 7, 5, 4, 2, 1, 0)	(98, 7, 4, 3, 2, 1, 0)
(49, 6, 5, 4, 0)	(99, 7, 5, 4, 0)
(50, 4, 3, 2, 0)	(100, 8, 7, 2, 0)

What are the conditions on $\mathbf{M}$ that give a full-period generator, and thereby prove that the polynomial with coefficients c_i is primitive? Evidently we must have

$$\mathbf{M}^{(2^n-1)} = \mathbf{1} \tag{7.5.8}$$

where $\mathbf{1}$ is the identity matrix. This states that the period, or some multiple of it, is $2^n - 1$. But the only possible such multiples are integers that divide $2^n - 1$. To rule these out, and ensure a full period, we need only check that

$$\mathbf{M}^{q_k} \neq \mathbf{1}, \qquad q_k \equiv (2^n - 1)/f_k \tag{7.5.9}$$

for every prime factor f_k of $2^n - 1$. (This is exactly the logic behind the tests of the matrix $\mathbf{T}$ that we described, but did not justify, in §7.1.2.)

It may at first sight seem daunting to compute the humongous powers of $\mathbf{M}$ in equations (7.5.8) and (7.5.9). But, by the method of repeated squaring of $\mathbf{M}$, each such power takes only about n (a number like 32 or 64) matrix multiplies. And, since all the arithmetic is done modulo 2, there is no possibility of overflow! The conditions (7.5.8) and (7.5.9) are in fact an efficient way to test a polynomial for primitiveness. The following code implements the test. Note that you must customize the constants in the constructor for your choice of n (called `N` in the code), in particular the prime factors of $2^n - 1$. The case $n = 32$ is shown. Other than that customization, the code as written is valid for $n \leq 64$. The input to the test is the "serial number," as defined above following equation (7.5.4), of the polynomial to be tested. After declaring an instance of the `Primpolytest` structure, you can repeatedly call its `test()` method to test multiple polynomials. To make `Primpolytest` entirely self-contained, matrices are implemented as linear arrays, and the structure builds from scratch the few matrix operations that it needs. This is inelegant, but effective.

primpolytest.h

```
struct Primpolytest {
```
Test polynomials over the integers mod 2 for primitiveness.
```
    Int N, nfactors;
    VecUllong factors;
    VecInt t,a,p;

    Primpolytest() : N(32), nfactors(5), factors(nfactors), t(N*N),
        a(N*N), p(N*N) {
```
        Constructor. The constants are specific to 32-bit LFSRs.
```
        Ullong factordata[5] = {3,5,17,257,65537};
        for (Int i=0;i<nfactors;i++) factors[i] = factordata[i];
    }

    Int ispident() {
```
        Utility to test if p is the identity matrix.
```
        Int i,j;
        for (i=0; i<N; i++) for (j=0; j<N; j++) {
            if (i == j) { if (p[i*N+j] != 1) return 0; }
            else {if (p[i*N+j] != 0) return 0; }
        }
        return 1;
    }

    void mattimeseq(VecInt &a, VecInt &b) {
```
        Utility for a *= b on matrices a and b.
```
        Int i,j,k,sum;
        VecInt tmp(N*N);
        for (i=0; i<N; i++) for (j=0; j<N; j++) {
            sum = 0;
            for (k=0; k<N; k++) sum += a[i*N+k] * b[k*N+j];
            tmp[i*N+j] = sum & 1;
        }
        for (k=0; k<N*N; k++) a[k] = tmp[k];
    }

    void matpow(Ullong n) {
```
        Utility for matrix p = a^n by successive squares.
```
        Int k;
```

```
        for (k=0; k<N*N; k++) p[k] = 0;
        for (k=0; k<N; k++) p[k*N+k] = 1;
        while (1) {
            if (n & 1) mattimeseq(p,a);
            n >>= 1;
            if (n == 0) break;
            mattimeseq(a,a);
        }
    }

    Int test(Ullong n) {
    Main test routine. Returns 1 if the polynomial with serial number n (see text) is primitive,
    0 otherwise.
        Int i,k,j;
        Ullong pow, tnm1, nn = n;
        tnm1 = ((Ullong)1 << N) - 1;
        if (n > (tnm1 >> 1)) throw("not a polynomial of degree N");
        for (k=0; k<N*N; k++) t[k] = 0;          Construct the update matrix in t.
        for (i=1; i<N; i++) t[i*N+(i-1)] = 1;
        j=0;
        while (nn) {
            if (nn & 1) t[j] = 1;
            nn >>= 1;
            j++;
        }
        t[N-1] = 1;
        for (k=0; k<N*N; k++) a[k] = t[k];       Test that t^tnm1 is the identity matrix.
        matpow(tnm1);
        if (ispident() != 1) return 0;
        for (i=0; i<nfactors; i++) {             Test that the t to the required submulti-
            pow = tnm1/factors[i];                   ple powers is not the identity matrix.
            for (k=0; k<N*N; k++) a[k] = t[k];
            matpow(pow);
            if (ispident() == 1) return 0;
        }
        return 1;
    }
};
```

It is straightforward to generalize this method to $n > 64$ or to prime moduli p other than 2. If $p^n > 2^{64}$, you'll need a multiword binary representation of the integers $p^n - 1$ and its quotients with its prime factors, so that `matpow` can still find powers by successive squares. Note that the computation time scales roughly as $O(n^4)$, so $n = 64$ is fast, while $n = 1024$ would be rather a long calculation.

Some random primitive polynomials for $n = 32$ bits (giving their serial numbers as decimal values) are 2046052277, 1186898897, 221421833, 55334070, 1225518245, 216563424, 1532859853, 1735381519, 2049267032, 1363072601, and 130420448. Some random ones for $n = 64$ bits are 926773948609480634, 3195735403700392248, 4407129700254524327, 256457582706860311, 5017679982664373343, and 1723461400905116882.

Given a matrix $\mathbf{M}$ that satisfies equations (7.5.8) and (7.5.9), there are some related matrices that also satisfy those relations. An example is the inverse of $\mathbf{M}$, which you can easily verify as

$$\mathbf{M}^{-1} = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 1 \\ 1 & c_1 & c_2 & \dots & c_{n-2} & c_{n-1} \end{bmatrix} \tag{7.5.10}$$

This is the update rule that backs up a state $\mathbf{a}'$ to its predecessor state $\mathbf{a}$. You can easily convert (7.5.10) to a prescription analogous to equation (7.5.1) or to Figure 7.5.1(a).

Another matrix satisfying the relations that guarantee a full period is the transpose of the